

Lab exercises

Part 01 —  Machine Learning Foundations · 01-ml-foundations/lab/EXERCISES.md

Lab — ML Foundations

Work top to bottom. Each exercise has a **stated deliverable**; if you can't produce it, you haven't finished. Solutions are intentionally not provided — the checks tell you when you're right.

1. Warm-up: read the code you were given (45 min)

Run both scripts in `../code/`. Then:

- **1a.** In `linear_models_from_scratch.py`, set `lr=5.0` and `epochs=200`. Explain the output in one sentence.
- **1b.** Replace the stable `sigmoid` with the naive `1/(1+np.exp(-z))` and evaluate it at `z = -800`. What warning appears, and why does the stable version avoid it?
- **1c.** Set `l2=0.5` on the linear model. Report the learned weights vs. the true `[3, -2]`. Which direction did they move, and why is that the point?

Deliverable: three sentences, one per part.

2. Implement softmax regression (2h)

Extend `LogisticRegressionGD` to `K` classes.

- Softmax: $p_k = \exp(z_k) / \sum_j \exp(z_j)$ — subtract $\max(z)$ before exponentiating or you will overflow.
- Loss: categorical cross-entropy.
- Gradient: $(1/n) X^T (P - Y_{\text{onehot}})$.

Checks: - Gradient check passes at $< 1e-5$. - On `sklearn.datasets.load_digits()` you reach ≥ 0.92 test accuracy. - Setting `K=2` reproduces your binary logistic results to 3 decimal places.

3. The leakage hunt (1.5h)

Take any tabular dataset (`sklearn.datasets.fetch_openml("adult")` works, or bring your own).

- **3a.** Build an honest pipeline; record CV ROC-AUC.
- **3b.** Inject three different leaks: (i) scale before splitting, (ii) add a feature derived from the target, (iii) duplicate 10% of rows across the split. Record each score.
- **3c.** For each leak, write the *symptom* you would have noticed in a code review.

Deliverable: a 4-row table (honest + 3 leaks) and your three symptoms.

4. Threshold economics (1h)

Using the imbalanced dataset from `evaluation_playbook.py`, assume: - A false negative (missed fraud) costs \$500. - A false positive (investigating a good customer) costs \$8.

Compute total expected cost across thresholds $0.01 \rightarrow 0.99$ and find the optimum.

Checks: - Your optimal threshold is **well below 0.5**. Explain in one sentence why that had to be true. - Plot cost vs. threshold; the curve should be convex-ish with a clear minimum.

5. Learning curves and diagnosis (1.5h)

Plot train and validation error against training-set size (10%, 20%, ... 100%) for: - (a) a degree-1 polynomial fit on non-linear data, - (b) a degree-15 polynomial fit on the same data, - (c) a well-tuned gradient-boosted model.

Deliverable: three plots and, for each, a one-word diagnosis: `bias-limited`, `variance-limited`, or `balanced`. Then answer: *for which of these would collecting $10\times$ more data help?*

6. Beat the baseline honestly (3h) — the capstone

Pick a real dataset (Kaggle Titanic, California Housing, or a work dataset).

Produce a single notebook containing, in order: 1. Problem framing: **what decision changes** because of this prediction? 2. A trivial baseline and its score. 3. A leakage-free preprocessing `Pipeline`. 4. Three models: logistic/linear $\rightarrow$ random forest $\rightarrow$ gradient boosting. 5. Cross-validated comparison with error bars. 6. A calibration curve for the winner. 7. Error analysis: the 20 worst predictions, grouped into ≥ 3 named failure categories. 8. A "do not trust this model when..." paragraph. 9. **One** test-set evaluation, run last.

Check: hand the notebook to someone else. If they can reproduce your headline number by running top-to-bottom, and they can name one situation where your model fails, you passed.

7. Stretch: nested cross-validation (2h)

Implement nested CV (outer 5-fold for estimation, inner 3-fold for tuning). Compare its score to a "tune-then-report-the-best-CV-score" approach on the same data.

Check: the naive approach is **optimistically biased**. Quantify the gap in AUC points. This gap is the tabular-ML version of benchmark contamination — see Module 15.